

Java Collection Interfaces

Twelve Demonstration Programs

Program 1: CollectionInterfaceDemo

Question

Write a Java program to demonstrate common operations of the Collection interface, including adding, searching, removing, iterating, and clearing elements.

Steps for Execution

1. Open a Java editor or Command Prompt.
2. Save the source code as `CollectionInterfaceDemo.java`.
3. Compile and run it with the commands on the final page for this program.
4. Observe the collection operations and their results.

Algorithm

1. Create a collection of fruits and add three values.
2. Create a second collection and add two more fruits.
3. Add all values from the second collection to the first.
4. Display the collection, membership checks, and size.
5. Remove Banana, then remove all values from the second collection.
6. Check whether the collection is empty and print its elements with an iterator.
7. Clear the collection, display it, and stop.

UML Class Diagram

CollectionInterfaceDemo
+main(args: String[]): void
Uses: Collection<String>, ArrayList<String>, Iterator<String>

CODE: SRC

```
import java.util.ArrayList;
import java.util.Collection;
import java.util.Iterator;

public class CollectionInterfaceDemo {
    public static void main(String[] args) {

        Collection<String> fruits = new ArrayList<>();

        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Mango");

        Collection<String> moreFruits = new ArrayList<>();
        moreFruits.add("Orange");
        moreFruits.add("Grapes");

        fruits.addAll(moreFruits);

        System.out.println("Fruits: " + fruits);
        System.out.println("Contains Mango: " + fruits.contains("Mango"));
        System.out.println("Contains all: " + fruits.containsAll(moreFruits));
        System.out.println("Size: " + fruits.size());

        fruits.remove("Banana");
        System.out.println("After remove: " + fruits);

        fruits.removeAll(moreFruits);
        System.out.println("After removeAll: " + fruits);

        System.out.println("Is empty: " + fruits.isEmpty());

        Iterator<String> it = fruits.iterator();

        System.out.print("Elements: ");
        while (it.hasNext()) {
            System.out.print(it.next() + " ");
        }

        fruits.clear();
        System.out.println("\nAfter clear: " + fruits);
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac CollectionInterfaceDemo.java
java CollectionInterfaceDemo
```

Output

```
Fruits: [Apple, Banana, Mango, Orange, Grapes]
Contains Mango: true
Contains all: true
Size: 5
After remove: [Apple, Mango, Orange, Grapes]
After removeAll: [Apple, Mango]
Is empty: false
Elements: Apple Mango
After clear: []
```

Program 2: DequeInterfaceDemo

Question

Write a Java program to demonstrate the Deque interface using an ArrayDeque. Add and offer values at both ends, inspect the first and last values, and remove values from both ends.

Steps for Execution

1. Open a Java editor or Command Prompt.
2. Save the source code as `DequeInterfaceDemo.java`.
3. Compile and run it with the commands on the final page for this program.
4. Observe the collection operations and their results.

Algorithm

1. Create an empty deque of strings.
2. Add B at the front and C at the back, then display it.
3. Offer A at the front and D at the back, then display the deque.
4. Peek at the first and last values.
5. Remove one value from each end and display the deque.
6. Poll the remaining values from both ends and display the final deque.

UML Class Diagram

DequeInterfaceDemo
+main(args: String[]): void
Uses: Deque<String>, ArrayDeque<String>

CODE: SRC

```
import java.util.ArrayDeque;
import java.util.Deque;

public class DequeInterfaceDemo {
    public static void main(String[] args) {

        Deque<String> deque = new ArrayDeque<>();

        deque.addFirst("B");
        deque.addLast("C");

        System.out.println("Deque: " + deque);

        deque.offerFirst("A");
        deque.offerLast("D");

        System.out.println("After offers: " + deque);

        System.out.println("First element: " + deque.peekFirst());
        System.out.println("Last element: " + deque.peekLast());

        System.out.println("Removed first: " + deque.removeFirst());
        System.out.println("Removed last: " + deque.removeLast());

        System.out.println("Deque: " + deque);

        System.out.println("Polled first: " + deque.pollFirst());
        System.out.println("Polled last: " + deque.pollLast());

        System.out.println("Final Deque: " + deque);
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac DequeInterfaceDemo.java
java DequeInterfaceDemo
```

Output

```
Deque: [B, C]
After offers: [A, B, C, D]
First element: A
Last element: D
Removed first: A
Removed last: D
Deque: [B, C]
Polled first: B
Polled last: C
Final Deque: []
```

Program 3: IteratorInterfaceDemo

Question

Write a Java program to demonstrate the Iterator interface. Traverse a list of subjects, remove a subject while iterating, and display the remaining subjects using `forEachRemaining`.

Steps for Execution

1. Open a Java editor or Command Prompt.
2. Save the source code as `IteratorInterfaceDemo.java`.
3. Compile and run it with the commands on the final page for this program.
4. Observe the collection operations and their results.

Algorithm

1. Create a list of subjects and add Java, DSA, OS, and CN.
2. Create an iterator and traverse the list, printing each subject.
3. When the iterator reaches OS, remove it using `Iterator.remove`.
4. Display the updated list.
5. Create a new iterator and print the remaining subjects with `forEachRemaining`.

UML Class Diagram

IteratorInterfaceDemo
+main(args: String[]): void
Uses: List<String>, ArrayList<String>, Iterator<String>

CODE: SRC

```
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

public class IteratorInterfaceDemo {
    public static void main(String[] args) {

        List<String> subjects = new ArrayList<>();

        subjects.add("Java");
        subjects.add("DSA");
        subjects.add("OS");
        subjects.add("CN");

        Iterator<String> it = subjects.iterator();

        System.out.println("Subjects:");

        while (it.hasNext()) {
            String subject = it.next();

            System.out.println(subject);

            if (subject.equals("OS")) {
                it.remove();
            }
        }

        System.out.println("After removing OS: " + subjects);

        it = subjects.iterator();

        System.out.println("Remaining subjects:");

        it.forEachRemaining(subject -> System.out.println(subject));
    }
}
```


Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac IteratorInterfaceDemo.java  
java IteratorInterfaceDemo
```

Output

Subjects:

Java

DSA

OS

CN

After removing OS: [Java, DSA, CN]

Remaining subjects:

Java

DSA

CN

Program 4: ListInterfaceDemo

Question

Write a Java program to demonstrate the List interface, including indexed insertion, retrieval, replacement, removal, searching, sublists, and sorting.

Steps for Execution

1. Open a Java editor or Command Prompt.
2. Save the source code as `ListInterfaceDemo.java`.
3. Compile and run it with the commands on the final page for this program.
4. Observe the collection operations and their results.

Algorithm

1. Create a list and add four names, including Kiran twice.
2. Display the list and insert Sita at index 2.
3. Retrieve the value at index 1 and replace it with Rahul.
4. Remove the value at index 2.
5. Display the first and last index of Kiran and a sublist.
6. Sort the names in natural order and display the result.

UML Class Diagram

ListInterfaceDemo
+main(args: String[]): void
Uses: List<String>, ArrayList<String>, Comparator<String>

CODE: SRC

```
import java.util.ArrayList;
import java.util.List;
import java.util.Comparator;

public class ListInterfaceDemo {
    public static void main(String[] args) {

        List<String> names = new ArrayList<>();

        names.add("Ravi");
        names.add("Kiran");
        names.add("Arun");
        names.add("Kiran");

        System.out.println("Names: " + names);

        names.add(2, "Sita");
        System.out.println("After inserting Sita: " + names);

        System.out.println("Element at index 1: " + names.get(1));

        names.set(1, "Rahul");
        System.out.println("After replacing: " + names);

        names.remove(2);
        System.out.println("After removing index 2: " + names);

        System.out.println("First Kiran index: " + names.indexOf("Kiran"));
        System.out.println("Last Kiran index: " + names.lastIndexOf("Kiran"));

        System.out.println("SubList: " + names.subList(1, 3));

        names.sort(Comparator.naturalOrder());
        System.out.println("Sorted names: " + names);
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac ListInterfaceDemo.java  
java ListInterfaceDemo
```

Output

```
Names: [Ravi, Kiran, Arun, Kiran]  
After inserting Sita: [Ravi, Kiran, Sita, Arun, Kiran]  
Element at index 1: Kiran  
After replacing: [Ravi, Rahul, Sita, Arun, Kiran]  
After removing index 2: [Ravi, Rahul, Arun, Kiran]  
First Kiran index: 3  
Last Kiran index: 3  
SubList: [Rahul, Arun]  
Sorted names: [Arun, Kiran, Rahul, Ravi]
```

Program 5: ListIteratorInterfaceDemo

Question

Write a Java program to demonstrate forward and backward traversal with a ListIterator and to modify a list using set, add, and remove.

Steps for Execution

1. Open a Java editor or Command Prompt.
2. Save the source code as `ListIteratorInterfaceDemo.java`.
3. Compile and run it with the commands on the final page for this program.
4. Observe the collection operations and their results.

Algorithm

1. Create a list containing Java, Python, C, and C++.
2. Use a ListIterator to traverse forward, displaying each index and value.
3. Use the same iterator to traverse backward.
4. Create a new iterator, replace Java with Java Programming, and display the list.
5. Add HTML after the replaced value and display the list.
6. Move back to HTML, remove it, and display the final list.

UML Class Diagram

```
classDiagram
    class ListIteratorInterfaceDemo {
        +main(args: String[]): void
    }
    ListIteratorInterfaceDemo --> List : Uses: List<String>
    ListIteratorInterfaceDemo --> ArrayList : Uses: ArrayList<String>
    ListIteratorInterfaceDemo --> ListIterator : Uses: ListIterator<String>
```

CODE: SRC

```
import java.util.ArrayList;
import java.util.List;
import java.util.ListIterator;

public class ListIteratorInterfaceDemo {
    public static void main(String[] args) {

        List<String> languages = new ArrayList<>();

        languages.add("Java");
        languages.add("Python");
        languages.add("C");
        languages.add("C++");

        ListIterator<String> it = languages.listIterator();

        System.out.println("Forward Traversal:");

        while (it.hasNext()) {
            System.out.println(
                "Index: " + it.nextIndex() +
                ", Value: " + it.next()
            );
        }

        System.out.println("Backward Traversal:");

        while (it.hasPrevious()) {
            System.out.println(
                "Index: " + it.previousIndex() +
                ", Value: " + it.previous()
            );
        }

        it = languages.listIterator();
        it.next();
        it.set("Java Programming");

        System.out.println("After set: " + languages);

        it.add("HTML");

        System.out.println("After add: " + languages);

        it.previous();
        it.remove();

        System.out.println("After remove: " + languages);
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac ListIteratorInterfaceDemo.java
java ListIteratorInterfaceDemo
```

Output

```
Forward Traversal:
Index: 0, Value: Java
Index: 1, Value: Python
Index: 2, Value: C
Index: 3, Value: C++
Backward Traversal:
Index: 3, Value: C++
Index: 2, Value: C
Index: 1, Value: Python
Index: 0, Value: Java
After set: [Java Programming, Python, C, C++]
After add: [Java Programming, HTML, Python, C, C++]
After remove: [Java Programming, Python, C, C++]
```

Program 6: MapInterfaceDemo

Question

Write a Java program to demonstrate the Map interface using a HashMap. Insert and retrieve product prices, remove an entry, inspect keys, values, and entries, and clear the map.

Steps for Execution

1. Open a Java editor or Command Prompt.
2. Save the source code as `MapInterfaceDemo.java`.
3. Compile and run it with the commands on the final page for this program.
4. Observe the collection operations and their results.

Algorithm

1. Create a map from product names to prices.
2. Add Pen, Book, and Bag with their prices.
3. Display the map and retrieve Book's price.
4. Remove Pen and display the map.
5. Check for Bag and the value 50.
6. Display the key set, values, entries, size, and empty status.
7. Clear the map and display its final state.

UML Class Diagram

```
classDiagram
    class MapInterfaceDemo {
        +main(args: String[]): void
    }
    MapInterfaceDemo "0" -- "*" Map : Uses: Map<String, Integer>, HashMap<String, Integer>
```


CODE: SRC

```
import java.util.HashMap;
import java.util.Map;

public class MapInterfaceDemo {
    public static void main(String[] args) {

        Map<String, Integer> products = new HashMap<>();

        products.put("Pen", 20);
        products.put("Book", 50);
        products.put("Bag", 100);

        System.out.println("Products: " + products);

        System.out.println("Price of Book: " + products.get("Book"));

        products.remove("Pen");
        System.out.println("After removing Pen: " + products);

        System.out.println("Contains Bag: " + products.containsKey("Bag"));
        System.out.println("Contains value 50: " + products.containsValue(50));

        System.out.println("Keys: " + products.keySet());
        System.out.println("Values: " + products.values());
        System.out.println("Entries: " + products.entrySet());

        System.out.println("Size: " + products.size());
        System.out.println("Is empty: " + products.isEmpty());

        products.clear();

        System.out.println("After clear: " + products);
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac MapInterfaceDemo.java
java MapInterfaceDemo
```

Output

Note: HashMap does not guarantee iteration order. The order of products, keys, values, and entries in this sample may differ when the program runs.

```
Products: {Book=50, Pen=20, Bag=100}
Price of Book: 50
After removing Pen: {Book=50, Bag=100}
Contains Bag: true
Contains value 50: true
Keys: [Book, Bag]
Values: [50, 100]
Entries: [Book=50, Bag=100]
Size: 2
Is empty: false
After clear: {}
```

Program 7: NavigableMapInterfaceDemo

Question

Write a Java program to demonstrate the NavigableMap interface using a TreeMap. Query neighboring keys, retrieve and remove the first and last entries, and display the map in descending order.

Steps for Execution

1. Open a Java editor or Command Prompt.
2. Save the source code as `NavigableMapInterfaceDemo.java`.
3. Compile and run it with the commands on the final page for this program.
4. Observe the collection operations and their results.

Algorithm

1. Create an empty navigable map and add five employee entries.
2. Display the map and query lower, floor, ceiling, and higher keys.
3. Display the first and last entries.
4. Remove the first entry with `pollFirstEntry` and display the map.
5. Remove the last entry with `pollLastEntry` and display the map.
6. Display the remaining entries in descending key order.

UML Class Diagram

```
classDiagram
    class NavigableMapInterfaceDemo {
        +main(args: String[]): void
    }
    NavigableMapInterfaceDemo --> NavigableMap : Uses: NavigableMap<Integer, String>, TreeMap<Integer, String>
```

CODE: SRC

```
import java.util.NavigableMap;
import java.util.TreeMap;

public class NavigableMapInterfaceDemo {
    public static void main(String[] args) {

        NavigableMap<Integer, String> employees = new TreeMap<>();

        employees.put(101, "Arun");
        employees.put(105, "Rahul");
        employees.put(110, "Sita");
        employees.put(115, "Vijay");
        employees.put(120, "Kiran");

        System.out.println("Employees: " + employees);

        System.out.println("Lower key than 110: " + employees.lowerKey(110));
        System.out.println("Floor key of 110: " + employees.floorKey(110));
        System.out.println("Ceiling key of 112: " + employees.ceilingKey(112));
        System.out.println("Higher key than 110: " + employees.higherKey(110));

        System.out.println("First Entry: " + employees.firstEntry());
        System.out.println("Last Entry: " + employees.lastEntry());

        System.out.println("Removed First Entry: " + employees.pollFirstEntry());
        System.out.println("Map: " + employees);

        System.out.println("Removed Last Entry: " + employees.pollLastEntry());
        System.out.println("Map: " + employees);

        System.out.println("Descending Map: " + employees.descendingMap());
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac NavigableMapInterfaceDemo.java
java NavigableMapInterfaceDemo
```

Output

```
Employees: {101=Arun, 105=Rahul, 110=Sita, 115=Vijay, 120=Kiran}
Lower key than 110: 105
Floor key of 110: 110
Ceiling key of 112: 115
Higher key than 110: 115
First Entry: 101=Arun
Last Entry: 120=Kiran
Removed First Entry: 101=Arun
Map: {105=Rahul, 110=Sita, 115=Vijay, 120=Kiran}
Removed Last Entry: 120=Kiran
Map: {105=Rahul, 110=Sita, 115=Vijay}
Descending Map: {115=Vijay, 110=Sita, 105=Rahul}
```

Program 8: NavigableSetInterfaceDemo

Question

Write a Java program to demonstrate the NavigableSet interface using a TreeSet. Find lower, floor, ceiling, and higher values, remove the first and last values, and display the set in descending order.

Steps for Execution

1. Open a Java editor or Command Prompt.
2. Save the source code as `NavigableSetInterfaceDemo.java`.
3. Compile and run it with the commands on the final page for this program.
4. Observe the collection operations and their results.

Algorithm

1. Create a navigable set and add five marks.
2. Display the set and query lower, floor, ceiling, and higher values.
3. Remove and display the first value.
4. Remove and display the last value.
5. Display the remaining values in descending order.

UML Class Diagram

```
classDiagram
    class NavigableSetInterfaceDemo {
        +main(args: String[]): void
    }
    NavigableSetInterfaceDemo --> NavigableSet : Uses: NavigableSet<Integer>, TreeSet<Integer>
```

CODE: SRC

```
import java.util.NavigableSet;
import java.util.TreeSet;

public class NavigableSetInterfaceDemo {
    public static void main(String[] args) {

        NavigableSet<Integer> marks = new TreeSet<>();

        marks.add(25);
        marks.add(40);
        marks.add(55);
        marks.add(70);
        marks.add(85);

        System.out.println("Marks: " + marks);

        System.out.println("Lower than 55: " + marks.lower(55));
        System.out.println("Floor of 55: " + marks.floor(55));
        System.out.println("Ceiling of 60: " + marks.ceiling(60));
        System.out.println("Higher than 55: " + marks.higher(55));

        System.out.println("First removed: " + marks.pollFirst());
        System.out.println("After pollFirst: " + marks);

        System.out.println("Last removed: " + marks.pollLast());
        System.out.println("After pollLast: " + marks);

        System.out.println("Descending Set: " + marks.descendingSet());
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac NavigableSetInterfaceDemo.java
java NavigableSetInterfaceDemo
```

Output

```
Marks: [25, 40, 55, 70, 85]
Lower than 55: 40
Floor of 55: 55
Ceiling of 60: 70
Higher than 55: 70
First removed: 25
After pollFirst: [40, 55, 70, 85]
Last removed: 85
After pollLast: [40, 55, 70]
Descending Set: [70, 55, 40]
```


Program 9: QueueInterfaceDemo

Question

Write a Java program to demonstrate the Queue interface using a LinkedList. Add and offer tasks, inspect the head with element and peek, and remove tasks with remove and poll.

Steps for Execution

1. Open a Java editor or Command Prompt.
2. Save the source code as `QueueInterfaceDemo.java`.
3. Compile and run it with the commands on the final page for this program.
4. Observe the collection operations and their results.

Algorithm

1. Create an empty queue of tasks.
2. Add Study, Exercise, and Practice.
3. Display the queue, offer Revision, and display it again.
4. Read the head using element and peek.
5. Remove the head using remove and display the queue.
6. Poll the next task and display the queue.

UML Class Diagram

```
classDiagram
    class QueueInterfaceDemo {
        +main(args: String[]): void
    }
    QueueInterfaceDemo --> Queue : Uses: Queue<String>
    QueueInterfaceDemo --> LinkedList : Uses: LinkedList<String>
```

CODE: SRC

```
import java.util.LinkedList;
import java.util.Queue;

public class QueueInterfaceDemo {
    public static void main(String[] args) {

        Queue<String> tasks = new LinkedList<>();

        tasks.add("Study");
        tasks.add("Exercise");
        tasks.add("Practice");

        System.out.println("Tasks: " + tasks);

        tasks.offer("Revision");
        System.out.println("After offer: " + tasks);

        System.out.println("First task using element: " + tasks.element());
        System.out.println("First task using peek: " + tasks.peek());

        System.out.println("Removed task: " + tasks.remove());
        System.out.println("Queue after remove: " + tasks);

        System.out.println("Polled task: " + tasks.poll());
        System.out.println("Queue after poll: " + tasks);
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac QueueInterfaceDemo.java
java QueueInterfaceDemo
```

Output

```
Tasks: [Study, Exercise, Practice]
After offer: [Study, Exercise, Practice, Revision]
First task using element: Study
First task using peek: Study
Removed task: Study
Queue after remove: [Exercise, Practice, Revision]
Polled task: Exercise
Queue after poll: [Practice, Revision]
```

Program 10: SetInterfaceDemo

Question

Write a Java program to demonstrate the Set interface using a HashSet. Add city names, observe duplicate handling, remove a city, check membership and size, and clear the set.

Steps for Execution

1. Open a Java editor or Command Prompt.
2. Save the source code as `SetInterfaceDemo.java`.
3. Compile and run it with the commands on the final page for this program.
4. Observe the collection operations and their results.

Algorithm

1. Create an empty set of cities.
2. Add Hyderabad, Chennai, Delhi, and Hyderabad again.
3. Display the set to observe that duplicate values are stored only once.
4. Remove Chennai and display the set.
5. Check whether Delhi is present, display the size, and check whether the set is empty.
6. Clear the set, then display it and check whether it is empty.

UML Class Diagram

```
classDiagram
    class SetInterfaceDemo {
        +main(args: String[]): void
    }
    SetInterfaceDemo "0" -- "*" Set : Uses: Set<String>, HashSet<String>
```

CODE: SRC

```
import java.util.HashSet;
import java.util.Set;

public class SetInterfaceDemo {
    public static void main(String[] args) {

        Set<String> cities = new HashSet<>();

        cities.add("Hyderabad");
        cities.add("Chennai");
        cities.add("Delhi");
        cities.add("Hyderabad");

        System.out.println("Cities: " + cities);

        cities.remove("Chennai");
        System.out.println("After removing Chennai: " + cities);

        System.out.println("Contains Delhi: " + cities.contains("Delhi"));
        System.out.println("Number of cities: " + cities.size());
        System.out.println("Is empty: " + cities.isEmpty());

        cities.clear();

        System.out.println("After clear: " + cities);
        System.out.println("Is empty: " + cities.isEmpty());
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac SetInterfaceDemo.java
java SetInterfaceDemo
```

Output

Note: HashSet does not guarantee iteration order. The order of cities in this sample may differ when the program runs.

```
Cities: [Delhi, Hyderabad, Chennai]
After removing Chennai: [Delhi, Hyderabad]
Contains Delhi: true
Number of cities: 2
Is empty: false
After clear: []
Is empty: true
```

Program 11: SortedMapInterfaceDemo

Question

Write a Java program to demonstrate the SortedMap interface using a TreeMap. Display its sorted entries, first and last keys, head, tail, and submaps, and comparator.

Steps for Execution

1. Open a Java editor or Command Prompt.
2. Save the source code as `SortedMapInterfaceDemo.java`.
3. Compile and run it with the commands on the final page for this program.
4. Observe the collection operations and their results.

Algorithm

1. Create a sorted map from student names to marks.
2. Insert five student entries.
3. Display the map, first key, and last key.
4. Display the head map, tail map, and submap around Sita.
5. Display the map comparator.

UML Class Diagram

```
SortedMapInterfaceDemo
+main(args: String[]): void
Uses: SortedMap<String, Integer>, TreeMap<String, Integer>
```

CODE: SRC

```
import java.util.SortedMap;
import java.util.TreeMap;

public class SortedMapInterfaceDemo {
    public static void main(String[] args) {

        SortedMap<String, Integer> students = new TreeMap<>();

        students.put("Kiran", 85);
        students.put("Arun", 75);
        students.put("Rahul", 90);
        students.put("Sita", 80);
        students.put("Vijay", 95);

        System.out.println("Sorted Map: " + students);

        System.out.println("First Key: " + students.firstKey());
        System.out.println("Last Key: " + students.lastKey());

        System.out.println("Head Map: " + students.headMap("Sita"));
        System.out.println("Tail Map: " + students.tailMap("Sita"));
        System.out.println("Sub Map: " + students.subMap("Arun", "Vijay"));

        System.out.println("Comparator: " + students.comparator());
    }
}
```


Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac SortedMapInterfaceDemo.java  
java SortedMapInterfaceDemo
```

Output

```
Sorted Map: {Arun=75, Kiran=85, Rahul=90, Sita=80, Vijay=95}  
First Key: Arun  
Last Key: Vijay  
Head Map: {Arun=75, Kiran=85, Rahul=90}  
Tail Map: {Sita=80, Vijay=95}  
Sub Map: {Arun=75, Kiran=85, Rahul=90, Sita=80}  
Comparator: null
```

Program 12: SortedSetInterfaceDemo

Question

Write a Java program to demonstrate the SortedSet interface using a TreeSet. Display its sorted values, first and last values, head, tail, and subset, and comparator.

Steps for Execution

1. Open a Java editor or Command Prompt.
2. Save the source code as `SortedSetInterfaceDemo.java`.
3. Compile and run it with the commands on the final page for this program.
4. Observe the collection operations and their results.

Algorithm

1. Create a sorted set of names.
2. Add five names in an unsorted order.
3. Display the set, first name, and last name.
4. Display the head set, tail set, and subset around Kiran.
5. Display the set comparator.

UML Class Diagram

```
SortedSetInterfaceDemo
+main(args: String[]): void
Uses: SortedSet<String>, TreeSet<String>
```

CODE: SRC

```
import java.util.TreeSet;
import java.util.SortedSet;

public class SortedSetInterfaceDemo {
    public static void main(String[] args) {

        SortedSet<String> names = new TreeSet<>();

        names.add("Vijay");
        names.add("Arun");
        names.add("Kiran");
        names.add("Sita");
        names.add("Rahul");

        System.out.println("Sorted Names: " + names);

        System.out.println("First Name: " + names.first());
        System.out.println("Last Name: " + names.last());

        System.out.println("Head Set: " + names.headSet("Kiran"));
        System.out.println("Tail Set: " + names.tailSet("Kiran"));
        System.out.println("Sub Set: " + names.subSet("Arun", "Sita"));

        System.out.println("Comparator: " + names.comparator());
    }
}
```

Compilation and Execution

Compile and run the program from the directory containing the source file:

```
javac SortedSetInterfaceDemo.java  
java SortedSetInterfaceDemo
```

Output

```
Sorted Names: [Arun, Kiran, Rahul, Sita, Vijay]  
First Name: Arun  
Last Name: Vijay  
Head Set: [Arun]  
Tail Set: [Kiran, Rahul, Sita, Vijay]  
Sub Set: [Arun, Kiran, Rahul]  
Comparator: null
```